

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Д. О. Шевяков
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №9

Настройка системы визуализации данных
для задач управления смарт-управления

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание	3
3 Ход выполнения задания.....	4
4 Вывод.....	7
ПРИЛОЖЕНИЕ А	8

1 Цель работы

Целью лабораторной работы является получение практических навыков визуализации телеметрических данных системы «умный дом» на основе данных, накопленных в базе MongoDB.

2 Задание

В соответствии с методическими указаниями для лабораторной работы №9 требуется построить график по данным из базы, реализовать API для выдачи рядов и оформить визуализацию с настройками внешнего вида.

3 Ход выполнения задания

В ходе выполнения лабораторной работы сохранены функции мониторинга, управления, логирования и анализа из предыдущих лабораторных работ, после чего добавлен модуль визуализации данных. На сервере реализован маршрут `GET /chart/temperature-series`, который формирует два массива: метки времени `labels` и значения температуры `values`, полученные из коллекции `Temperature` в MongoDB. Дополнительно поддержан параметр `limit` для ограничения количества точек на графике. На рисунке 1 изображена главная страница ЛР9 со ссылкой на модуль визуализации.

Мониторинг
Температура: 26.2 °C
Свет: ВКЛ, яркость 10%
Замок: ОТКРЫТ
Обогреватель: ВЫКЛ (порог 25 °C)

Управление (с валидацией на сервере)
Температура
Примеры: 23.5 или 23,5

Свет
☐ enabled — слово: on, off, true, false, да, нет ... Яркость — только цифры 0-100.

Замок
Строка из словаря: закрыт, открыт, true, false, ...

Анализ данных


```
{
  "loggerEnabled": true,
  "temperature": {
    "mean": 21.018860759493645,
    "max": 27.7,
    "count": 790
  },
  "heaterSnapshots": {
    "on": 22,
    "off": 17
  }
}
```

Визуализация
Линейный график по журналу температуры, плавные кривые (tension), подписи осей.
[chart.html — график](#)

Рисунок 1 - Скриншот главной страницы ЛР9: блок «Визуализация» и ссылка на `chart.html`.

Для отображения графика создана отдельная страница `chart.html`, на которой подключена библиотека Chart.js (CDN) и клиентский скрипт `chart.js`. Скрипт запрашивает данные из `/chart/temperature-series`, обрабатывает временные метки и строит линейный график температуры с настраиваемыми параметрами оформления: заливка под кривой, сглаживание линии, подписи осей и заголовок диаграммы. На рисунке 2 изображён построенный график температуры.

[← На главную](#)

ЛР9 — визуализация журнала температуры

Данные из MongoDB, GET /chart/temperature-series?limit=...

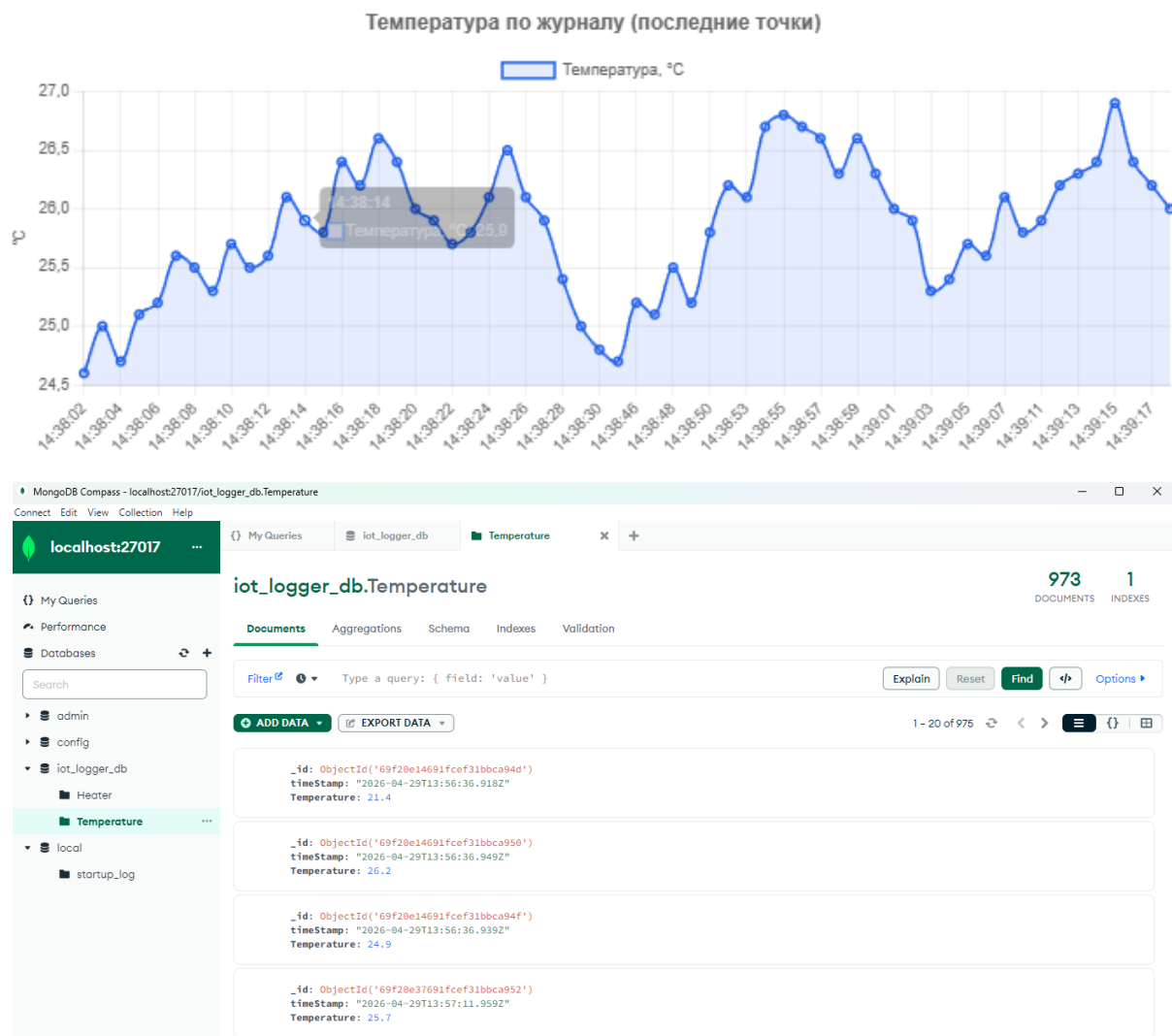


Рисунок 2 - Скриншот страницы `chart.html`: линейный график температуры по данным журнала MongoDB`.

Корректность обмена между клиентом и сервером подтверждается в инструментах разработчика: при открытии `chart.html` выполняется запрос к маршруту визуализации и возвращается JSON с массивами `labels` и `values`. На рисунке 3 изображена вкладка Network с запросом `/chart/temperature-series` и телом ответа.

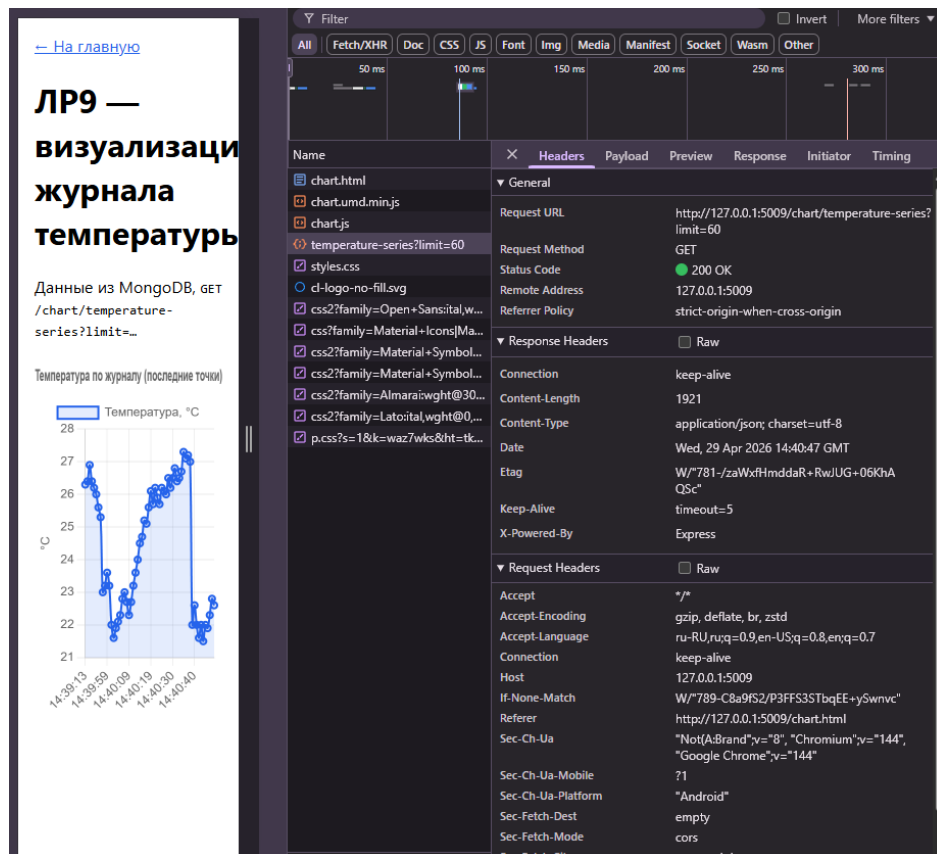


Рисунок 3 - Скриншот Network: запрос `GET /chart/temperature-series` и JSON-ответ с данными графика.

4 Вывод

В результате выполнения лабораторной работы реализована подсистема визуализации данных на основе ранее накопленного журнала MongoDB. Построение графика температуры выполнено через отдельный API-маршрут и страницу с Chart.js, что обеспечивает наглядное представление динамики параметров и соответствует требованиям лабораторной работы №9.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
// chart.js
async function loadChart() {
  const canvas = document.getElementById("tempChart");
  const r = await fetch("/chart/temperature-series?limit=60");
  const { labels, values } = await r.json();

  if (!labels.length) {
    const ctx = canvas.getContext("2d");
    if (ctx) {
      ctx.font = "16px system-ui";
      ctx.fillStyle = "#64748b";
      ctx.fillText("Нет данных в MongoDB — подождите накопления
опросом /connect/temperature", 8, 40);
    }
    return;
  }

  const shortLabels = labels.map((s) => (s.length > 19 ? s.slice(11, 19) : s));

  new Chart(canvas, {
    type: "line",
    data: {
      labels: shortLabels,
      datasets: [
        {
          label: "Температура, °C",
          data: values,
```

```

        borderColor: "#2563eb",
        backgroundColor: "rgba(37, 99, 235, 0.12)",
        borderWidth: 2,
        fill: true,
        tension: 0.35,
        pointRadius: 3,
        pointHoverRadius: 6
    }
]
},
options: {
    responsive: true,
    maintainAspectRatio: false,
    interaction: { intersect: false, mode: "index" },
    plugins: {
        title: {
            display: true,
            text: "Температура по журналу (последние точки)",
            font: { size: 16, weight: "600" }
        },
        legend: { position: "top" }
    },
    scales: {
        x: {
            grid: { color: "rgba(0,0,0,0.06)" },
            ticks: { maxRotation: 45, minRotation: 0 }
        },
        y: {
            grid: { color: "rgba(0,0,0,0.08)" },
            title: { display: true, text: "°C" }

```

```

        }
    }
}
});
}

```

```

loadChart().catch((e) => {
    document.body.appendChild(document.createTextNode(String(e)));
});

```

// агрегаты

```

app.get("/analysis/stats", async (_req, res) => {
    const temperature = await logger.getTemperatureMeanAndMax();
    const heaterLog = await logger.getHeaterOnOffCounts();
    res.json({
        loggerEnabled: logger.isEnabled(),
        temperature,
        heaterSnapshots: heaterLog
    });
});

```

```

void (async function main() {
    app.listen(PORT, () => {
        console.log(`http://127.0.0.1:${PORT}/ — ЛР8 (анализ: GET
/analysis/stats)`);
    });
    await logger.init();
    if (!logger.isEnabled()) {
        console.warn("MongoDB недоступна — анализ и журнал будут пустыми,
UI доступен");
    }
}

```

```

    }
  })().catch((e) => {
    console.error(e);
    process.exit(1);
  });

//logger mongodb
import { MongoClient, type Db } from "mongodb";

const DEFAULT_URI = process.env.MONGODB_URI ??
"mongodb://127.0.0.1:27017";

export class IoTLogger {
  private client: MongoClient | null = null;
  private db: Db | null = null;
  private lastLoggedTemperature: number | null = null;
  private enabled = false;

  async init(dbName = "iot_logger_db"): Promise<void> {
    try {
      this.client = new MongoClient(DEFAULT_URI, {
        serverSelectionTimeoutMS: 4000,
        connectTimeoutMS: 4000
      });
      await this.client.connect();
      this.db = this.client.db(dbName);
      this.enabled = true;
      console.log(`[IoTLogger] MongoDB: ${DEFAULT_URI}, БД
«${dbName}»`);
    }
  }
}

```

```

    } catch (err) {
        console.warn("[IoTLogger] подключение к MongoDB не удалось,
логирование отключено:", err);
        this.enabled = false;
        this.db = null;
        if (this.client) {
            try {
                await this.client.close();
            } catch {
                /* ignore */
            }
            this.client = null;
        }
    }
}

```

```

isEnabled(): boolean {
    return this.enabled && this.db !== null;
}

```

*/** Коллекция Temperature: без дублей подряд одинакового значения (как в пособии). */*

```

async insertTemperature(value: number): Promise<void> {
    if (!this.db || !this.enabled) return;
    if (this.lastLoggedTemperature === value) {
        console.log("[IoTLogger] температура не изменилась — запись не
дублируется");
        return;
    }
    this.lastLoggedTemperature = value;

```

```

    await this.db.collection("Temperature").insertOne({
      timeStamp: new Date().toISOString(),
      Temperature: value
    });
  }

  /** Вторая коллекция — снимки состояния обогревателя. */
  async insertHeaterSnapshot(power: string): Promise<void> {
    if (!this.db || !this.enabled) return;
    await this.db.collection("Heater").insertOne({
      timeStamp: new Date().toISOString(),
      power
    });
  }

  async close(): Promise<void> {
    await this.client?.close();
  }
}

//обмен данных между датчиком и исполнителем
export class HeaterController extends Thing {
  private power: "On" | "Off" = "Off";

  constructor(id: string, name: string, private readonly switchOnBelowC: number)
  {
    super(id, name, true);
  }
}

```

```
/** Включение при температуре строго ниже порога (как в пособии, порог 25 °C). */
```

```
autoPower(temperatureC: number): void {  
    this.power = temperatureC < this.switchOnBelowC ? "On" : "Off";  
    console.log(  
        `[HeaterController.autoPower] ${this.id}: t=${temperatureC.toFixed(1)}°C,  
порог=${this.switchOnBelowC}°C -> ${this.power}`  
    );  
}
```

```
getPower(): "On" | "Off" {  
    return this.power;  
}
```

```
protected emulate(): void {  
    /* состояние задаётся только autoPower, без случайной эмуляции */  
}
```

```
protected buildPayload(): Record<string, unknown> {  
    return {  
        id: this.id,  
        name: this.name,  
        heaterPower: this.power,  
        switchOnBelowC: this.switchOnBelowC  
    };  
}
```

```
getStatus(): string {  
    return `${this.name}: ${this.power === "On" ? "ВКЛ" : "ВЫКЛ"}`;  
}
```

```
}
```

```
// валидация validation.ts
```

```
function showBad(url) {  
  const out = document.getElementById("badDemoOut");  
  fetch(url)  
    .then(async (r) => {  
      const text = await r.text();  
      try {  
        const j = JSON.parse(text);  
        out.textContent = `HTTP ${r.status}\n${JSON.stringify(j, null, 2)}`;  
      } catch {  
        out.textContent = `HTTP ${r.status}\n${text}`;  
      }  
    })  
    .catch((e) => {  
      out.textContent = String(e);  
    });  
}
```

```
document.getElementById("badTemp").addEventListener("click", () => {  
  showBad("/command/temperature?value=abc");  
});
```

```
document.getElementById("badLight").addEventListener("click", () => {  
  showBad("/command/light?enabled=maybe&brightness=50");  
});
```

```
document.getElementById("badLock").addEventListener("click", () => {  
  showBad("/command/lock?locked=скоpee_нет");
```



```
});
```

```
// app.ts
```

```
import express from "express";
```

```
import path from "node:path";
```

```
import { TemperatureSensor, LightController, SmartLock } from "./things";
```

```
const app = express();
```

```
const PORT = 5000;
```

```
const temp = new TemperatureSensor("sensor-1", "Температурный датчик", 22.5);
```

```
const light = new LightController("light-1", "Свет в гостиной");
```

```
const lock = new SmartLock("lock-1", "Замок входной двери");
```

```
light.setLight(true, 65);
```

```
lock.setLocked(true);
```

```
app.use((req, _res, next) => {
```

```
  if (req.path.startsWith("/connect") || req.path.startsWith("/command")) {
```

```
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.path}`);
```

```
  }
```

```
  next();
```

```
});
```

```
app.use(express.static(path.join(process.cwd(), "public")));
```

```
app.get("/connect/temperature", (_req, res) => res.json(temp.connect()));
```

```
app.get("/connect/light", (_req, res) => res.json(light.connect()));
```

```
app.get("/connect/lock", (_req, res) => res.json(lock.connect()));
```

```

app.get("/command/temperature", (req, res) => {
  const q = req.query as Record<string, string | undefined>;
  const result = temp.applyCommand(q);
  if (!result.ok) {
    return res.status(400).json(result);
  }
  return res.json({ ok: true, state: temp.snapshot() });
});

```

```

app.get("/command/light", (req, res) => {
  const q = req.query as Record<string, string | undefined>;
  const result = light.applyCommand(q);
  if (!result.ok) {
    return res.status(400).json(result);
  }
  return res.json({ ok: true, state: light.snapshot() });
});

```

```

app.get("/command/lock", (req, res) => {
  const q = req.query as Record<string, string | undefined>;
  const result = lock.applyCommand(q);
  if (!result.ok) {
    return res.status(400).json(result);
  }
  return res.json({ ok: true, state: lock.snapshot() });
});

```

```

app.listen(PORT, () => {
  console.log(`http://127.0.0.1:${PORT}/`);

```

```
});
```

```
//things.ts
```

```
export type CommandResult = { ok: true } | { ok: false; error: string };
```

```
export abstract class Thing {
```

```
  constructor(
```

```
    public id: string,
```

```
    public name: string,
```

```
    public isOnline: boolean = true
```

```
  ) {}
```

```
  protected abstract emulate(): void
```

```
  protected abstract buildPayload(): Record<string, unknown>;
```

```
  connect(): Record<string, unknown> {
```

```
    console.log(`[Thing.connect] ${this.constructor.name} (${this.id})`);
```

```
    this.emulate();
```

```
    return this.buildPayload();
```

```
  }
```

```
  /** Снимок состояния без эмуляции (после команды). */
```

```
  snapshot(): Record<string, unknown> {
```

```
    return this.buildPayload();
```

```
  }
```

```
  abstract getStatus(): string;
```

```
}
```

```

export interface IParent {
    render(statuses: string[]): string;
}

export interface IChild {
    render(statuses: string[]): string;
}

export class MainControlUnit {
    constructor(
        public readonly things: Thing[],
        public readonly dataStore: DeviceDataStore
    ) {}

    registerThing(thing: Thing): void {
        console.log(`[MainControlUnit.registerThing] ${thing.id}`);
        this.things.push(thing);
    }

    collectMonitoringData(): void {
        console.log(`[MainControlUnit.collectMonitoringData]`);
        for (const thing of this.things) {
            const line = `[${new Date().toISOString()}] ${thing.getStatus()}`
            this.dataStore.saveRecord(thing, line)
        }
    }

    getAllStatuses(): string[] {
        console.log(`[MainControlUnit.getAllStatuses]`);
        return this.things.map((t) => t.getStatus());
    }
}

```

```
}
```

```
export class DeviceDataStore {
```

```
  private readonly dataByThingId = new Map<string, string[]>();
```

```
  saveRecord(thing: Thing, payload: string): void {
```

```
    console.log(`[DeviceDataStore.saveRecord] ${thing.id}`);
```

```
    const list = this.dataByThingId.get(thing.id) ?? [];
```

```
    list.push(payload);
```

```
    this.dataByThingId.set(thing.id, list);
```

```
  }
```

```
  getRecords(thingId: string): string[] {
```

```
    console.log(`[DeviceDataStore.getRecords] ${thingId}`);
```

```
    return this.dataByThingId.get(thingId) ?? [];
```

```
  }
```

```
}
```

```
export class TemperatureSensor extends Thing {
```

```
  private currentTemperatureC: number;
```

```
  constructor(id: string, name: string, initialTemperatureC: number) {
```

```
    super(id, name, true);
```

```
    this.currentTemperatureC = initialTemperatureC;
```

```
  }
```

```
  setTemperature(valueC: number): void {
```

```
    console.log(`[TemperatureSensor.setTemperature] ${this.id}`);
```

```
    this.currentTemperatureC = valueC;
```

```
}
```

```
getTemperature(): number {  
    return this.currentTemperatureC;  
}
```

```
/** IP4: параметры из query (аналог request.args). */
```

```
applyCommand(query: Record<string, string | undefined>): CommandResult {  
    const raw = query.value ?? query.temperature;  
    if (raw === undefined || raw === "") {  
        return { ok: false, error: "Ожидается параметр value (или temperature)" };  
    }  
    const n = Number(raw);  
    if (Number.isNaN(n)) {  
        return { ok: false, error: "value должно быть числом" };  
    }  
    this.setTemperature(n);  
    console.log(`[TemperatureSensor.applyCommand] ${this.id} -> ${n}`);  
    return { ok: true };  
}
```

```
protected emulate(): void {  
    const delta = (Math.random() - 0.5) * 1.2;  
    this.currentTemperatureC = Math.round((this.currentTemperatureC + delta) *  
10) / 10;  
}
```

```
protected buildPayload(): Record<string, unknown> {  
    return {  
        id: this.id,
```

```

    name: this.name,
    value: this.currentTemperatureC,
    unit: "°C"
  };
}

```

```

getStatus(): string {
  console.log(`[TemperatureSensor.getStatus] ${this.id}`);
  return `${this.name}: ${this.currentTemperatureC.toFixed(1)} °C`
}
}

```

```

export class LightController extends Thing {
  private brightnessPercent: number = 0;
  private isEnabled: boolean = false;

```

```

  setLight(isEnabled: boolean, brightnessPercent: number): void {
    console.log(`[LightController.setLight] ${this.id}`);
    this.isEnabled = isEnabled;
    this.brightnessPercent = Math.max(0, Math.min(100, brightnessPercent))
  }

```

```

  applyCommand(query: Record<string, string | undefined>): CommandResult {
    const enabledRaw = query.enabled ?? query.on;
    const brightRaw = query.brightness ?? query.brightnessPercent;
    if (enabledRaw === undefined || brightRaw === undefined || brightRaw ===
    "") {
      return { ok: false, error: "Ожидаются enabled (true/false) и brightness (0–
    100)" };
    }
  }
}

```

```

    }
    const enabled = enabledRaw === "true" || enabledRaw === "1";
    const brightness = Number(brightRaw);
    if (Number.isNaN(brightness)) {
        return { ok: false, error: "brightness должно быть числом" };
    }
    this.setLight(enabled, brightness);
    console.log(`[LightController.applyCommand] ${this.id} enabled=${enabled}
brightness=${brightness}`);
    return { ok: true };
}

protected emulate(): void {
    const delta = Math.floor((Math.random() - 0.5) * 12);
    this.brightnessPercent = Math.max(0, Math.min(100, this.brightnessPercent +
delta));
}

protected buildPayload(): Record<string, unknown> {
    return {
        id: this.id,
        name: this.name,
        enabled: this.isEnabled,
        brightnessPercent: this.brightnessPercent
    };
}

getStatus(): string {
    console.log(`[LightController.getStatus] ${this.id}`);

```



```

        return `${this.name}: ${this.isEnabled ? "BKЛ" : "БЫКЛ"},
яркость=${this.brightnessPercent}%`;
    }
}

```

```

export class SmartLock extends Thing{
    private isLocked: boolean = true;

```

```

    setLocked(value: boolean): void {
        console.log(`[SmartLock.setLocked] ${this.id}`);
        this.isLocked = value;
    }

```

```

    applyCommand(query: Record<string, string | undefined>): CommandResult {
        const raw = query.locked ?? query.lock;
        if (raw === undefined || raw === "") {
            return { ok: false, error: "Ожидается locked (true/false)" };
        }
        if (raw !== "true" && raw !== "false" && raw !== "1" && raw !== "0") {
            return { ok: false, error: "locked должно быть true или false" };
        }
        const locked = raw === "true" || raw === "1";
        this.setLocked(locked);
        console.log(`[SmartLock.applyCommand] ${this.id} locked=${locked}`);
        return { ok: true };
    }

```

```

    protected emulate(): void {
        if (Math.random() < 0.08) this.isLocked = !this.isLocked;
    }

```

```

protected buildPayload(): Record<string, unknown> {
  return {
    id: this.id,
    name: this.name,
    locked: this.isLocked
  };
}

getStatus(): string {
  console.log(`[SmartLock.getStatus] ${this.id}`);
  return `${this.name}: ${this.isLocked ? "ЗАКРЫТ" : "ОТКРЫТ"}`;
}
}

export class ParentInterface implements IParent {
  render(statuses: string[]): string {
    console.log("[ParentInterface.render]");
    return ["Интерфейс родителя (полный доступ)", ...statuses].join("\n");
  }
}

export class ChildInterface implements IChild {
  render(statuses: string[]): string {
    console.log("[ChildInterface.render]");
    return ["Интерфейс ребенка (ограничен)", ...statuses].join("\n");
  }
}

export function runDemo(): {

```

```

parentText: string;
childText: string;
sensorRecords: string[];
} {
  const store = new DeviceDataStore();
  const things: Thing[] = [];
  const mcu = new MainControlUnit(things, store);
  const temp = new TemperatureSensor("sensor-1", "Температурный датчик",
22.5);
  const light = new LightController("light-1", "Свет в гостиной");
  const lock = new SmartLock("lock-1", "Замок входной двери");
  light.setLight(true, 65);
  lock.setLocked(true);
  mcu.registerThing(temp);
  mcu.registerThing(light);
  mcu.registerThing(lock);
  mcu.collectMonitoringData();
  const statuses = mcu.getAllStatuses();
  const parentText = new ParentInterface().render(statuses);
  const childText = new ChildInterface().render(statuses);
  return {
    parentText,
    childText,
    sensorRecords: store.getRecords("sensor-1")
  };
}

```

```
//script.js
```

```

const CONNECT = {
  temperature: "/connect/temperature",

```

```
    light: "/connect/light",  
    lock: "/connect/lock",  
};
```

```
async function fetchJson(url) {  
    const response = await fetch(url);  
    if (!response.ok) {  
        throw new Error(`HTTP ${response.status} для ${url}`);  
    }  
    return response.json();  
}
```

```
//index.js  
function $(id) {  
    return document.getElementById(id);  
}
```

```
async function updateTemperature() {  
    const data = await fetchJson(CONNECT.temperature)  
    document.getElementById("tempValue").textContent = `${data.value}  
    ${data.unit}`  
}
```

```
async function updateLight() {  
    const data = await fetchJson(CONNECT.light);  
    $("lightEnabled").textContent = data.enabled ? "ВКЛ" : "ВЫКЛ";  
    $("lightBrightness").textContent = String(data.brightnessPercent);  
}
```

```
async function updateLock() {
```

```

const data = await fetchJson(CONNECT.lock);
$("lockState").textContent = data.locked ? "ЗАКРЫТ" : "ОТКРЫТ";
}
async function tick() {
  await Promise.all([updateTemperature(), updateLight(), updateLock()]);
}

```

```

tick();
setInterval(tick, 1000);

```

```

//index.html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>IoT Дом — ЛР4</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 42rem; margin: 1.5rem
auto; padding: 0 1rem; }
    section { border: 1px solid #ddd; border-radius: 8px; padding: 0.75rem 1rem;
margin: 1rem 0; }
    h1 { font-size: 1.35rem; }
    code { background: #f4f4f4; padding: 0.1rem 0.35rem; border-radius: 4px; }
    .result { font-size: 0.9rem; color: #333; margin-top: 0.35rem; }
  </style>
</head>
<body>
  <h1>ЛР4 — мониторинг (ЛР3) + команды управления</h1>

```

```

<section>
  <h2>Мониторинг</h2>
  <p>Обновление раз в секунду через <code>GET /connect/...</code></p>
  <p><strong>Температура:</strong> <span id="tempValue">—</span></p>
  <p><strong>Свет:</strong> <span id="lightEnabled">—</span>, яркость
  <span id="lightBrightness">—</span>%</p>
  <p><strong>Замок:</strong> <span id="lockState">—</span></p>
</section>

```

```

<section>
  <h2>Управление</h2>
  <p>Отправка на <code>GET /command/...</code> с параметрами в строке
  запроса.</p>

```

```

<div>
  <h3>Температура</h3>
  <input id="cmdTemp" type="text" placeholder="например 23.5" />
  <button id="btnTemp" type="button">Отправить</button>
  <div class="result" id="cmdTempResult"></div>
</div>

```

```

<div>
  <h3>Свет</h3>
  <label><input id="cmdLightOn" type="checkbox" checked />
  Включено</label>
  <input id="cmdLightBright" type="number" min="0" max="100" value="65"
  />
  <button id="btnLight" type="button">Отправить</button>
  <div class="result" id="cmdLightResult"></div>
</div>

```

```

<div>
  <h3>Замок</h3>
  <label><input id="cmdLockLocked" type="checkbox" checked />
  Закрыт</label>
  <button id="btnLock" type="button">Отправить</button>
  <div class="result" id="cmdLockResult"></div>
</div>
</section>

<script src="/script.js"></script>
<script src="/index.js"></script>
<script src="/command.js"></script>
</body>
</html>

```

```

//command.js
async function sendCommand(url) {
  const response = await fetch(url);
  const data = await response.json();
  if (!response.ok) {
    throw new Error(JSON.stringify(data));
  }
  return data;
}

function setResult(id, text) {
  const el = document.getElementById(id);
  if (el) el.textContent = text;
}

```

```

document.getElementById("btnTemp").addEventListener("click", async () => {
    const value = document.getElementById("cmdTemp").value.trim();
    try {
        const data = await sendCommand(
            `/command/temperature?value=${encodeURIComponent(value)}`
        );
        setResult("cmdTempResult", JSON.stringify(data));
    } catch (e) {
        setResult("cmdTempResult", String(e.message));
    }
});

```

```

document.getElementById("btnLight").addEventListener("click", async () => {
    const enabled = document.getElementById("cmdLightOn").checked;
    const brightness = document.getElementById("cmdLightBright").value;
    const qs = new URLSearchParams({
        enabled: String(enabled),
        brightness: String(brightness)
    });
    try {
        const data = await sendCommand(`/command/light?${qs.toString()}`);
        setResult("cmdLightResult", JSON.stringify(data));
    } catch (e) {
        setResult("cmdLightResult", String(e.message));
    }
});

```

```

document.getElementById("btnLock").addEventListener("click", async () => {
    const locked = document.getElementById("cmdLockLocked").checked;

```



```
const qs = new URLSearchParams({ locked: String(locked) });
try {
  const data = await sendCommand(`/command/lock?${qs.toString()}`);
  setResult("cmdLockResult", JSON.stringify(data));
} catch (e) {
  setResult("cmdLockResult", String(e.message));
}
});
```